

# Lab 1 Build Guide: Proxmox VM + ContainerLab + Data Model

This guide walks through the entire Lab 1 setup from bare Proxmox to a validated spine-leaf data model. It is structured as a single continuous session you can follow on camera for a YouTube walkthrough.

## What We Are Building

A 6-node VXLAN/EVPN spine-leaf fabric running FRR inside ContainerLab on a dedicated Proxmox VM. By the end, you will have a validated YAML data model describing the fabric's intent, with Pydantic catching any structural or referential errors before a single config touches a router.

## Prerequisites

You need a Proxmox host with enough spare resources for one small VM. The lab is lightweight. You also need an ISO for Ubuntu Server 24.04 LTS uploaded to your Proxmox storage (or use the built-in download feature under the ISO Images section of your storage).

## Part 1: Create the Proxmox VM

### VM Specifications

| Resource | Value                            | Why                                                         |
|----------|----------------------------------|-------------------------------------------------------------|
| RAM      | 4 GB                             | 6 FRR containers at ~80 MB each plus Docker and OS overhead |
| Disk     | 32 GB                            | Ubuntu base + Docker images + ContainerLab artifacts        |
| Network  | 1 vNIC on your management bridge | SSH access from your workstation                            |

| vCPUS |

These are modest on purpose. The entire lab runs comfortably on hardware that would struggle with a single Cisco CML node.

### Step-by-Step in the Proxmox UI

1. Click "Create VM" in the top right of the Proxmox web UI.
2. General tab: Give it a name like clab-nac and pick a VM ID. Leave the rest default.
3. OS tab: Select your Ubuntu 24.04 Server ISO from the storage dropdown. Guest OS type is Linux, version 6.x - 2.6 Kernel.
4. System tab: Change the BIOS to OVMF (UEFI) if you prefer, but SeaBIOS works fine. Add an EFI disk if you chose OVMF. Enable Qemu Agent so Proxmox can read the VM's IP after boot.
5. Disks tab: 32 GB on your preferred storage. VirtIO Block is the best performing option. Enable Discard if you are using thin provisioning.
6. CPU tab: 2 cores, type "host" for best performance. The "host" CPU type passes through your physical CPU features, which avoids emulation overhead.
7. Memory tab: 4096 MB. Uncheck ballooning unless you are tight on RAM across VMs and want Proxmox to reclaim unused memory.
8. Network tab: Select your management bridge (the same one your k3s node uses, likely vmbr0). Model VirtIO for best throughput. VLAN tag only if your management network requires it.

9. Confirm and create. Do not start it yet if you want to double-check the settings.

10. Start the VM and open the console. Walk through the Ubuntu installer. Choose "Ubuntu Server (minimized)" if offered. Set a username and enable OpenSSH server during install. You do not need LVM or any special partitioning for a lab VM.

After the install finishes and the VM reboots, note the IP address from the Proxmox summary page (the Qemu Agent reports it) or check from the console with `ip addr`.

## SSH In

From your workstation:

```
ssh your-username@<vm-ip>
```

Everything from here runs inside the VM over SSH.

## Part 2: Install the Toolchain

Run these commands in order. Each block is copy-pasteable as a single unit.

### System Updates

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl ca-certificates
```

## Docker

The convenience script from Docker's official site detects your distro and installs the latest stable release.

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
```

Log out and back in (or run `newgrp docker`) so the group membership takes effect. Verify with:

```
docker run --rm hello-world
```

If you see "Hello from Docker!", you are good.

## Kernel Modules for EVPN Dataplane

The VXLAN/EVPN dataplane requires the `vrf` and `vxlan` kernel modules. Ubuntu cloud images ship a minimal kernel that may not include them by default. Install the extra modules package and load them:

```
sudo apt install -y linux-modules-extra-$(uname -r)
sudo modprobe vrf
sudo modprobe vxlan
```

Make them persistent across reboots:

```
echo -e "vrf\nvxlan" | sudo tee /etc/modules-load.d/nac-lab.conf
```

Verify they loaded:

```
lsmod | grep -E 'vrf|vxlan'
```

You should see both `vrf` and `vxlan` in the output. Without these modules, the FRR control plane (BGP, OSPF) will work fine, but the VXLAN tunnels and VRF isolation that make the EVPN overlay functional will not come up.

## ContainerLab

```
sudo bash -c "$(curl -sL https://get.containerlab.dev)"
```

Verify the install:

```
containerlab version
```

You should see version 0.62 or later. ContainerLab pulls FRR images from the container registry on first deploy, so no separate image download is needed.

## Python and uv

Ubuntu 24.04 ships with Python 3.12. Verify it is present:

```
python3 --version
```

Install uv, the fast Python package manager:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Restart your shell or source the profile so uv is on your PATH:

```
source $HOME/.local/bin/env
uv --version
```

## Quick Verification

At this point you should have all four tools available:

```
docker --version
containerlab version
python3 --version
uv --version
```

If any of these fail, fix it before moving on. Every step after this depends on all four being present.

## Part 3: Initialize the Project

Create the project directory and initialize it with uv:

```
mkdir -p ~/network-as-code-labs
cd ~/network-as-code-labs
uv init --name iac-labs
```

This creates a pyproject.toml and a basic project structure. Now add the dependencies we need for Lab 1:

```
uv add "pydantic==2.11.4" "pyyaml==6.0.2" "rich==14.0.0" "types-pyyaml==6.0.12.20250402"
```

Notice the exact version pins. No ^ or ~ ranges. Every dependency is locked to a specific version so that the project builds identically on every machine. The uv.lock file records the full resolution and should be committed to Git.

## Initialize Git

This is a good time to set up version control. The entire point of Network as Code is that your network intent lives in a repository, versioned and reviewable like any other code.

```
git init
git branch -M main
```

Create a .gitignore so that Python artifacts, virtual environments, and secrets never end up in the repository:

```
cat > .gitignore << 'EOF'
# Python
__pycache__/*
*.py[cod]
*$py.class
*.so
*.egg-info/
dist/
build/
*.egg

# Virtual environments
.venv/
venv/
ENV/

# IDE
.idea/
.vscode/
*.swp
*.swo
*~

# Environment and secrets
.env
.env.local
.env.*.local

# OS
.DS_Store
Thumbs.db

# ContainerLab
clab-*/
*.bak

# Linter caches
.ruff_cache/
.mypy_cache/

# Test artifacts
htmlcov/
.coverage
.pytest_cache/
*.xml
reports/

# Generated configs
configs/
configs-ansible/
configs-tf/
.terraform/
*.tfstate
*.tfstate.backup
.terraform.lock.hcl
EOF
```

The .gitignore is worth pausing on. The .env entry is a security requirement. Device credentials, API tokens, and anything sensitive goes in environment variables or .env files that are excluded from the repository. The configs/ entries exclude generated output because the source of truth is the data model, not the rendered configs. The clab-\*/ entry excludes ContainerLab runtime state.

## Part 4: Explore the Data Model

Before deploying anything, walk through the YAML files that define the fabric's intent. This is the core teaching moment of Lab 1: the network exists as structured data before it exists as router configuration.

### Fabric-Wide Settings

```
cat data/fabric.yaml
```

This is the highest-level declaration. It says: "This fabric is called nac-spine-leaf, it uses ASN 65000, OSPF for the underlay, and BGP EVPN for the overlay." It also defines the three IP ranges that partition the address space across loopbacks, point-to-point links, and management interfaces.

## Device Inventory

```
cat data/topology.yaml
```

Six devices, each with a name, role, loopback, management IP, and ASN. The `route_reflector: true` flag on spine1 and spine2 is the intent declaration that drives the entire overlay peering topology. You do not manually specify BGP neighbor statements. You declare which devices are route reflectors and the automation (Lab 3) builds the peering from there.

## Underlay Links

```
cat data/underlay.yaml
```

Eight point-to-point /31 links connecting every spine to every leaf and border. OSPF area 0 with a 100 Gbps reference bandwidth so that 10G and 100G links get meaningful cost differentiation. The link definitions are bidirectional: each one names both the A-side and B-side device, interface, and IP.

## Overlay Configuration

```
cat data/overlay.yaml
```

Both spines as route reflectors with their loopbacks as cluster IDs. BGP EVPN plus IPv4 unicast address families. Standard 30/90 keepalive/holdtime timers.

## Services

```
cat data/services/vrfs.yaml
cat data/services/networks.yaml
cat data/services/interfaces.yaml
```

Three VRFs (PRODUCTION, DEVELOPMENT, MANAGEMENT), five network segments across them, and six host-facing interface assignments on the leafs and borders. Every network references a VRF by name. Every interface references VLANs by ID. These cross-references are validated by the schema.

## Defaults

```
cat data/defaults.yaml
```

Jumbo frames (9216 MTU) on fabric links, 1500 on management, head-end replication for BUM traffic, ARP suppression enabled. These are the values that apply when nothing is explicitly overridden.

## Part 5: Run the Validation

This is where the data model proves its value. The validation script loads every YAML file, parses it through its Pydantic schema, then runs cross-file checks.

```
uv run python validate.py
```

You should see all eight files pass individual validation, then cross-reference validation pass, followed by a summary table showing 6 devices, 8 links, 2 route reflectors, 3 VRFs, 5 networks, and 6 interface assignments.

The point here is that the data model is not just documentation. It is a machine-verifiable contract. Every file has a schema, and the schemas reference each other. A network segment points to a VRF by name. An interface assignment points to a device and a VLAN. The validator checks all of these cross-references in one pass. If something does not line up, you find out now instead of after pushing config to a router.

Lab 2 builds a full four-layer validation framework on top of this foundation, with rule IDs, pytest integration, and detailed error reporting. For now, seeing all checks pass confirms the data model is solid.

## Part 6: First Git Commit

The data model is built and validated. This is the right time for the first commit. Everything in the repository at this point is the foundation that every subsequent lab builds on.

Check what Git sees:

```
git status
```

You should see the data files, schemas, ContainerLab topology, validate.py, pyproject.toml, uv.lock, and .gitignore as untracked files. Stage and commit them:

```
git add .
git status
```

Review the staged files. You should not see any .env files, \_\_pycache\_\_/ directories, or .venv/ in the list. The .gitignore is doing its job.

```
git commit -m "Lab 1: data model, Pydantic schemas, and ContainerLab topology"
```

Check the log:

```
git log --oneline
```

One commit. The entire network intent for a 6-node spine-leaf fabric, validated and version-controlled. Every change from here forward is a diff against this baseline. When Lab 4 introduces CI/CD, this repository becomes the trigger for automated validation and deployment pipelines.

## Part 7: Deploy the ContainerLab Topology

Now bring up the actual routers. This creates six FRR containers connected in the spine-leaf topology.

```
sudo containerlab deploy --topo containerlab/topology.yaml
```

ContainerLab pulls the FRR image on first run (about 150 MB). After that it takes a few seconds to spin up the containers and wire the links.

Verify the topology is running:

```
sudo containerlab inspect --topo containerlab/topology.yaml
```

You should see all six nodes in the "running" state with their management IPs assigned.

## Connect to a Device

```
docker exec -it clab-nac-spine-leaf-spine1 vtysh
```

This drops you into FRR's VTY shell on spine1. At this point the routers have no configuration beyond their base interfaces. That is intentional. Lab 1 is about the data model. Labs 3 and 4 will generate and push configuration from this data model to these routers.

Type exit to leave vtysh, then exit again to leave the container.

## Tear Down (When Done)

```
sudo containerlab destroy --topo containerlab/topology.yaml
```

This removes all containers and virtual links but preserves the data on disk. You can redeploy at any time.

## Part 8: What We Proved

By the end of this lab you have demonstrated four things.

First, that network intent can be expressed as structured YAML data. The six files in data/ describe the entire fabric without a single router CLI command. This is the "single source of truth" concept.

Second, that schema validation catches errors before they reach the network. Overlapping subnets, undefined VRF references, duplicate loopbacks, interfaces on the wrong device role -- all caught at the data layer. The paper that inspired this series cites 80%+ of network issues coming from misconfigurations. A validated data model eliminates an entire class of those.

Third, that the gap between "data model" and "running routers" is bridgeable. The same topology defined in YAML is running as containers you can SSH into. Labs 2 through 7 close this gap progressively: validation, config generation, CI/CD pipelines, post-change testing, drift detection, and AI-assisted operations.

Fourth, that version control is part of the workflow from day one. The data model lives in Git. Every change is a commit. Every commit is a reviewable diff. This is the foundation that makes CI/CD possible in Lab 4. You cannot have automated pipelines without a repository to trigger them.

## Troubleshooting

ContainerLab fails with "permission denied": Make sure you are running with sudo. ContainerLab needs root to create network namespaces and virtual links.

Docker socket permission error after install: You need to log out and back in after `usermod -aG docker $USER`. Running `newgrp docker` in the current shell is a quick workaround.

FRR image pull fails: Check your VM has internet access. Run `docker pull quay.io/frrouting/frr:10.3.1` manually to see the full error. If you are behind a proxy, configure Docker's proxy settings in `/etc/docker/daemon.json`.

uv sync fails with Python version error: The project requires Python 3.11+. Ubuntu 24.04 ships 3.12, which works. If you are on an older Ubuntu, install Python 3.11+ from the deadsnakes PPA: `sudo add-apt-repository ppa:deadsnakes/ppa && sudo apt install python3.11`.

Validation passes but you expected it to fail: Make sure you saved the file after editing. YAML is whitespace-sensitive. Check that your edit did not accidentally fix a different issue (like removing a required field entirely, which would trigger a different error than the one you expected).